Visual Exploratory Data Analysis (VisEDA): PART A

Image Exploratory Data Analysis

A Guide to Image Dataset Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

Preface

The practice of Exploratory Data Analysis (EDA) has been a cornerstone of data science since its formalisation by John Tukey in 1977. For tabular data, the Python ecosystem offers excellent libraries — pandas-profiling, ydata-profiling, and SweetViz have transformed how analysts understand structured datasets. Yet for visual data — images, hyperspectral cubes, and point clouds — no equivalent toolkit existed. Researchers and practitioners working in computer vision, remote sensing, and 3D perception were left to write their own one-off scripts every time they encountered a new dataset.

VisEDA was created to fill that gap. It brings the same rigour and convenience of tabular EDA to visual data modalities, enabling researchers to deeply understand a dataset before training a model, diagnosing data quality issues, and communicating dataset properties clearly to collaborators.

This book documents VisEDA version 1.0.0 with a focus on the ImageEDA module, which is the most complete and production-ready component of the library. Subsequent chapters on HyperspectralEDA and PointCloudEDA will be added as those modules mature.

Isaac Osei Agyemang
May 2026

Table of Contents

Chapter 1 — Introduction

1.1 What is VisEDA?

VisEDA (Visual Exploratory Data Analysis) is an open-source Python library designed to help researchers, data scientists, and machine learning engineers perform comprehensive exploratory analysis on visual datasets. Unlike tabular EDA tools that operate on rows and columns, VisEDA understands images as structured arrays of pixels with spatial, photometric, and semantic properties.

The library operates at two levels. At the individual image level, it computes per-image statistics covering spatial properties, pixel distributions, colour characteristics, texture patterns, frequency content, and quality metrics. At the dataset level, it aggregates these statistics across thousands of images, revealing distributions, anomalies, class imbalances, and data quality issues that would be impossible to see by looking at individual images.

1.2 Why Image EDA Matters

Before training any computer vision model, understanding your dataset is not optional—it is essential. The following questions should be answered before a single training epoch begins:

- What is the distribution of image sizes? A mix of very small (e.g., 50×50 px) and very large (e.g., 500×500 px) images may require adaptive preprocessing.
- How balanced are the classes? An imbalance ratio of 10:1 between, for example, dogs and cats (i.e., *dogs and cats are the dataset classes*) will bias a classifier regardless of architecture.
- What fraction of images are blurry, overexposed, or corrupted? Poor-quality images degrade model performance silently.
- Are there near-duplicate images that would cause data leakage between train and validation splits?
- What are the dataset-level mean and standard deviation per channel? These are needed for normalisation in torchvision and similar frameworks.
- Are there systematic colour biases (e.g., all outdoor images skewing warm) that might cause the model to learn spurious correlations?

VisEDA answers all of these questions automatically, with visualisations that communicate the answers clearly.

1.3 Design Philosophy

- Zero required configuration. Point VisEDA at a directory and it works.
- Progressive disclosure. summary() gives you the numbers; plot() gives you the pictures; report() gives you a shareable document.
- Lazy dependencies. Heavy packages like scikit-image (for GLCM) are only imported when needed. The library works without them, simply skipping the analyses they power.
- Memory safety. Large datasets are handled through sampling and streaming; VisEDA does not load all images into RAM simultaneously.

1.4 Installation

Install the core library from PyPI:

```
pip install viseda
```

For full functionality including GLCM texture analysis:

```
pip install viseda scikit-image
```

1.5 Supported Data Formats

ImageEDA supports the following image formats: JPEG (.jpg, .jpeg), PNG (.png), BMP (.bmp), TIFF (.tif, .tiff), WebP (.webp), and GIF (.gif). Both 8-bit and 16-bit images are accepted; float images in the [0, 1] range are automatically converted to uint8.

Chapter 2 — Quick Start

2.1 Loading a Dataset from a Directory

The most common usage pattern is loading an image dataset organised into class sub-folders — the same structure used by PyTorch's ImageFolder and Keras's image_dataset_from_directory:

```
# Dataset structure:
# PetImages/
#   Cat/
#     0.jpg, 1.jpg, ...
#   Dog/
#     0.jpg, 1.jpg, ...

from viseda import ImageEDA

eda = ImageEDA(verbose=True)
eda.load("PetImages/", label_from_parent=True)
```

The label_from_parent=True argument tells VisEDA to use the parent folder name (Cat, Dog) as the label for each image. This is the most convenient option for standard classification datasets.

2.2 Getting a Summary

```
stats = eda.summary()

# Access individual sections
print(stats["inventory"])      # counts, formats, colour modes
print(stats["spatial"])        # height, width, aspect ratio, MP
print(stats["quality"])        # brightness, sharpness, noise, blur
print(stats["colour"])         # saturation, colour temperature
print(stats["texture"])        # GLCM features
print(stats["frequency"])      # FFT band energies
print(stats["duplicates"])     # exact and near-duplicate groups
print(stats["labels"])         # class distribution, imbalance ratio
```

2.3 Generating Visualisations

```
eda.plot()                        # full 6-row EDA dashboard
eda.plot_colour()                 # colour analysis dashboard
eda.plot_quality()                # quality metrics dashboard
eda.plot_texture()                # GLCM texture dashboard
eda.plot_samples(n=25, cols=5)    # random sample grid
eda.plot_class_samples(n_per_class=6)  # one row per class
eda.plot_average_image()          # dataset mean & variance image
eda.plot_channel_correlation()    # R/G/B scatter matrix
eda.plot_duplicates(mode="near")  # near-duplicate groups
```

To save plots to disk instead of displaying them interactively, pass save_path to any plot method:

```
eda.plot(save_path="dashboard.png", dpi=150)
eda.plot_colour(save_path="colour.png")
```

2.4 Getting Normalisation Statistics

Before training a neural network, images are normalised by subtracting the dataset mean and dividing by the standard deviation, per channel. VisEDA computes these automatically:

```
norm = eda.normalization_stats()
print(norm)
# {
#   "mean": [0.4889, 0.4523, 0.4177],  # R, G, B in [0, 1]
#   "std":  [0.2297, 0.2290, 0.2285]
# }

# Use directly in torchvision:
import torchvision.transforms as T
transform = T.Compose([
    T.ToTensor(),
    T.Normalize(mean=norm["mean"], std=norm["std"]),
])
```

2.5 Generating an HTML Report

```
eda.report("viseda_report.html")
# Open viseda_report.html in any browser
```

The report is entirely self-contained — it is a single HTML file with no external dependencies, safe to share via email or upload to a server.

2.6 Command-Line Interface

VisEDA also ships a CLI for quick dataset checks without writing Python:

```
# Full analysis with report and saved plots
viseda image path/to/dataset/ --label-from-parent --report out.html --plot

# Quick check on first 1000 images
viseda image path/to/dataset/ --max 1000 --quick

# Skip GLCM for faster loading
viseda image path/to/dataset/ --no-glcm --save-plots
```

Chapter 3 — The ImageEDA Class

3.1 Constructor

All configuration is set at construction time. You create one ImageEDA object per dataset analysis session.

```
from viseda import ImageEDA

eda = ImageEDA(
    verbose       = True,   # print progress messages
    max_images    = None,   # analyse all images (or set a limit)
    n_colors      = 8,      # dominant colour clusters to extract
    phash_threshold = 10,   # Hamming distance for near-duplicates
    blur_threshold  = 50.0, # Laplacian variance below = blurry
    compute_glcm    = True, # texture analysis (needs scikit-image)
    compute_freq    = True, # FFT frequency analysis
)
```

Parameter	Description
verbose	When True, prints a progress line every 5% of images loaded. Useful for large datasets to confirm the library is running.
max_images	Caps the number of images analysed. Useful for quick exploratory runs. Pass None to analyse the entire dataset.
n_colors	Number of dominant colour clusters extracted via K-Means. 6–10 is typical. Higher values capture more subtle palette variation.
phash_threshold	Maximum Hamming distance between perceptual hashes for two images to be considered near-duplicates. 0 = identical hashes only. 10 is a practical default that catches resized, slightly compressed, or recoloured copies.
blur_threshold	Laplacian variance threshold for blurry detection. Images with sharpness below this value are flagged as blurry. Natural photos typically score 100–5000; blurry ones score below 50.
compute_glcm	Whether to compute Grey-Level Co-occurrence Matrix texture features. Requires scikit-image. Set False for faster loading on large datasets where texture is not needed.
compute_freq	Whether to compute FFT frequency band energy ratios. Very fast; rarely needs to be disabled.

3.2 load() — Load from Directory or File List

The primary loading method. Walks a directory, discovers supported image files, computes all per-image statistics, and stores them internally.

```
eda.load(
    source           = "path/to/dataset/",
    labels           = None,           # optional {path: label} dict
    label_from_parent = False,          # infer label from folder name
    recursive        = True,           # recurse into sub-directories
)
```

Typical usage patterns:

```
# Pattern 1: class sub-folders (most common)
eda.load("ImageNet/train/", label_from_parent=True)

# Pattern 2: flat directory, no labels
eda.load("unlabelled_images/")

# Pattern 3: manual label mapping
labels = {"img001.jpg": "cat", "img002.jpg": "dog"}
eda.load("images/", labels=labels)

# Pattern 4: explicit list of paths
from pathlib import Path
paths = list(Path("images").glob("**/*.jpg"))[:500]
eda.load(paths)
```

Note: Corrupt or unreadable images are not silently dropped — they are flagged in the summary under inventory["corrupt_paths"] so you know exactly which files need attention.

3.3 load_arrays() — Load NumPy Arrays Directly

Use this when your images are already in memory as NumPy arrays, such as when loading from HDF5 files (e.g. ModelNet40), generating synthetic data for testing, or working in a pipeline that does not store images to disk.

```
import numpy as np

# Each array: HxW (grayscale) or HxWx3 (RGB)
# dtype: uint8 [0, 255] or float32 [0.0, 1.0]

arrays = [np.random.randint(0, 255, (256, 256, 3), dtype=np.uint8)
          for _ in range(100)]
labels = ["cat"] * 50 + ["dog"] * 50

eda = ImageEDA(verbose=False)
eda.load_arrays(arrays, labels=labels)
```

3.4 summary() — Full Statistics Dictionary

Returns a nested dictionary with all computed statistics. This is the primary programmatic interface — use it when you want to access specific numbers in your own code rather than looking at plots.

```
stats = eda.summary()

# The dictionary has seven top-level sections:
# stats["inventory"]   — counts, formats, colour modes
# stats["spatial"]     — height, width, aspect ratio, megapixels
# stats["pixel_stats"] — per-channel means and stds
# stats["quality"]     — brightness, sharpness, blur, exposure
# stats["colour"]      — saturation, temperature, greyscale detection
# stats["texture"]     — GLCM features (if compute_glcm=True)
# stats["frequency"]   — FFT band energy (if compute_freq=True)
# stats["duplicates"]  — exact and near-duplicate groups
# stats["labels"]      — class distribution, imbalance ratio
```

Every numeric field is a statistics dictionary with the same structure:

```
# Example: stats["spatial"]["height"]
{
    "min":    50.0,
    "max":    500.0,
    "mean":   361.26,
    "median": 375.0,
    "std":    67.8,
    "p25":    333.0,   # 25th percentile
    "p75":    407.0    # 75th percentile
}
```

Real results from the cats-and-dogs dataset (24,998 images):

```
stats["inventory"]["total"]          # 24998
stats["inventory"]["valid"]          # 24946
stats["inventory"]["corrupt"]        # 52
stats["quality"]["brightness"]["mean"] # 117.47 (out of 255)
stats["quality"]["blurry_count"]     # 1122 (4.5%)
stats["colour"]["colour_temp_distribution"]
# {"warm": 15076, "neutral": 8712, "cool": 1158}
stats["labels"]["label_distribution"]
# {"Cat": 12479, "Dog": 12467}
stats["labels"]["class_imbalance_ratio"] # 1.001 (perfectly balanced)
```

3.5 normalization_stats() — Training Normalisation Values

Computes the per-channel mean and standard deviation across the entire dataset, expressed in the [0, 1] range. These are the values you pass directly to torchvision.transforms.Normalize or equivalent frameworks when preparing images for neural network training.

The mean tells the network what "average" pixel looks like — subtracting it centres the data around zero, which helps gradient-based optimisers converge faster. The standard deviation tells the network how much variation to expect — dividing by it puts all channels on a similar scale, preventing one channel from dominating the loss.

```
norm = eda.normalization_stats()
# {"mean": [0.4889, 0.4523, 0.4177], "std": [0.2297, 0.2290, 0.2285]}

# PyTorch usage:
import torchvision.transforms as T
transform = T.Compose([
    T.Resize(224),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=norm["mean"], std=norm["std"]),
])

# TensorFlow / Keras usage:
mean = np.array(norm["mean"]) * 255.0
std  = np.array(norm["std"])  * 255.0
```

Interpretation: For the cats-and-dogs dataset, mean=[0.49, 0.45, 0.42] shows that images are slightly red-biased (R > B), consistent with warm-toned fur and outdoor backgrounds. std≈0.23 across all channels indicates moderate variation — not a flat, low-contrast dataset.

3.6 report() — HTML Report

Generates a complete, self-contained HTML report at the specified path. The report includes all sections from summary() formatted in a readable card layout, with a normalisation stats code snippet ready to copy. No internet connection is required to view it.

```
eda.report("dataset_report.html")
# Opens in any browser; safe to share via email
```

Chapter 4 — What Is Analysed

This chapter explains every metric VisEDA computes, what it means, how to interpret it, and what action it might suggest. Understanding these metrics is essential for making good use of the library.

4.1 Dataset Inventory

The inventory section answers the most basic questions about a dataset: how many images are there, what formats are they in, and are any of them broken?

Total / Valid / Corrupt Count

VisEDA attempts to read every file it discovers. A file is marked corrupt if OpenCV returns None when reading it, which happens with truncated JPEG files, zero-byte files, or files with a wrong extension. The corrupt_paths list lets you identify and remove or fix these files before training.

In the cats-and-dogs dataset, 52 of 24,998 files (0.2%) were corrupt — a level typical of web-scraped datasets.

Format Distribution

Tells you which file formats are present. A mixed-format dataset (e.g., some PNG, some JPEG) is fine for training, but it is worth knowing because PNG files are losslessly compressed (no compression artefacts) while JPEG files have compression artefacts that GLCM and frequency analysis will detect.

Colour Mode Distribution

Distinguishes RGB (3-channel), grayscale (1-channel), and RGBA (4-channel, with transparency) images. Many models expect 3-channel input; if your dataset contains grayscale images, you need to explicitly convert them (e.g., using torchvision's Grayscale transform or cv2.cvtColor).

4.2 Spatial Analysis

Spatial statistics describe the geometric properties of the images — their sizes and shapes.

Height and Width

Most deep learning models expect a fixed input size (e.g. 224×224 for ImageNet-pretrained networks). If your dataset has variable-size images, the height and width distributions tell you what resize target to choose. A narrow distribution (e.g., most images between 300–500 px) requires less aggressive resizing than a wide one (50 px to 4000 px).

In the cats-and-dogs dataset: mean height = 361 px, mean width = 405 px, with images ranging from 50×50 to 500×500 px. The large spike visible in the width histogram at 500 px is because that is the dataset's capping size — many images were cropped or resized to 500 px maximum during dataset creation.

Aspect Ratio

The ratio of width to height. A value of 1.0 is a square image. Values above 1.0 are landscape (wider than tall); values below 1.0 are portrait. The orientation distribution (landscape/square/portrait counts)

summarises this quickly. Aspect ratio matters because cropping strategies affect landscape and portrait images differently — a centre crop on a portrait image may cut off more content than on a square one.

Megapixels

Total pixel count (height × width) divided by 1,000,000. A dataset of 0.15 MP average images (roughly 375×400) contains far fewer pixels than one averaging 2 MP (roughly 1400×1400), which affects training speed and the level of detail the model can learn from.

4.3 Quality Metrics

Quality metrics identify images that may be problematic for training. A model trained on predominantly sharp, well-exposed images will learn differently from one trained on a mix of sharp and blurry images — and a dataset contaminated with corrupt-quality images produces unreliable benchmarks.

Brightness

The mean luminance of the image, computed as a weighted average of the R, G, and B channels using the ITU-R BT.601 formula: L = 0.299R + 0.587G + 0.114B. Range is 0 (pure black) to 255 (pure white). The human eye is most sensitive to green, hence the high weight on G.

A dataset with a very low mean brightness (e.g. below 80) may have been collected in poor lighting or at night — which may or may not be appropriate for your task. A high mean (above 200) suggests overexposed imagery. The cats-and-dogs dataset has mean brightness = 117 ± 30, indicating a naturally-lit, diverse collection.

Contrast

The standard deviation of the luminance across the pixels of a single image. A low contrast image (std < 20) is nearly uniform in brightness — think a blank sky or a white wall. A high contrast image (std > 80) has bright and dark regions side by side.

Low contrast in a dataset can indicate that images were poorly lit, severely compressed, or that the subjects lack visual variety. The cats-and-dogs dataset shows mean contrast = 57 — typical of natural photographs.

Sharpness (Laplacian Variance)

Sharpness is computed as the variance of the image after applying a Laplacian filter. The Laplacian is a second derivative operator — it produces large values at edges and near-zero values in flat regions. A sharp image has many strong edges; a blurry image has few. The variance of the Laplacian response therefore scales with the overall sharpness of the image.

The sharpness axis is plotted on a logarithmic scale because the distribution spans many orders of magnitude (from 1 for extremely blurry to 10,000 for razor-sharp). In the cats-and-dogs dataset, the mean sharpness is 867, and images below 50 are flagged as blurry. Out of 24,946 valid images, 1,122 (4.5%) were blurry — a notable quality issue worth investigating before training.

> **Tip:** To see which images are blurry, inspect [r for r in eda._records if r.is_blurry] and call eda.plot_samples() with a filtered list.

Noise Estimate

Estimated by subtracting a Gaussian-blurred version of the image from the original and measuring the standard deviation of the residual. A clean image has a small residual; a noisy image has a large one. This metric detects sensor noise (common in low-light photography), JPEG compression noise, and scan artefacts.

The cats-and-dogs dataset has mean noise = 6.97 — a typical level for JPEG-compressed web images. Values above 15 would indicate significant noise worth addressing with denoising preprocessing.

Overexposed and Underexposed Pixel Fraction

Overexposed pixels have luminance > 245 (near-white, clipped highlights). Underexposed pixels have luminance < 10 (near-black, crushed shadows). These fractions are computed per image and then averaged across the dataset.

A mean overexposed fraction of 2.67% means that on average, 2.67% of each image's pixels are blown out to white. This is normal for images with bright backgrounds or direct sunlight. If the fraction exceeds 20%, the images may be systematically overexposed.

The exposure scatter plot in plot_quality() shows each image as a point with underexposed fraction on the x-axis and overexposed fraction on the y-axis. Images near the origin are well-exposed; images far from the origin have problematic exposure.

JPEG Blockiness Score

JPEG compression works by dividing images into 8×8 pixel blocks and compressing each block independently. At low quality settings, visible discontinuities appear at the block boundaries — the "blockiness" artefact. VisEDA estimates this by measuring the mean absolute difference in pixel values across 8-pixel boundaries. A high score indicates heavy JPEG compression.

The cats-and-dogs dataset has a mean blockiness of 7.45 — moderate compression typical of images downloaded from the web at reasonable quality.

Pixel Entropy

Shannon entropy of the pixel value histogram, measured in bits. A completely uniform image (all pixels the same value) has entropy = 0 bits. A perfectly random image (all pixel values equally likely) has entropy = 8 bits (for uint8). Most natural images fall between 6–8 bits.

Very low entropy (below 4) suggests the image may be nearly blank, very dark, or highly compressed. The cats-and-dogs dataset has mean entropy = 7.35 bits — a healthy level indicating visually diverse, information-rich images.

Blurry Detection Flag

Each image receives a boolean is_blurry flag based on whether its Laplacian variance is below blur_threshold (default: 50). The blurry count and fraction in summary() aggregate these flags across the dataset. In plot_quality(), a bar chart shows the sharp/blurry split with exact counts.

4.4 Colour Analysis

Colour analysis goes beyond simple channel histograms to characterise the chromatic properties of the dataset as a whole.

Per-Channel Mean Distribution

Shows the distribution of per-image average pixel values, separately for the Red, Green, and Blue channels. When these distributions overlap heavily and are centred similarly, the dataset is colour-balanced. When one channel is consistently higher (e.g., Red peaks at 150 while Blue peaks at 100), there is a systematic colour cast that the model will learn.

In the cats-and-dogs dataset, Red mean ≈ 125, Green mean ≈ 115, Blue mean ≈ 107. The red-green-blue ordering (decreasing) is consistent with the warm colour temperature finding: animal fur and outdoor scenes in natural sunlight skew warm (more red, less blue).

Dominant Colour Palette

K-Means clustering is applied to a random sample of pixels from a sample of images to find the k most representative colours (default k=8). The result is a bar chart showing each colour (rendered in its actual colour) alongside the percentage of sampled pixels it represents.

This is one of the most visually intuitive analyses in VisEDA. For the cats-and-dogs dataset, the palette is dominated by browns, tans, greys, and near-blacks — the natural colours of cat and dog fur — with white and light grey representing backgrounds.

HSV Hue Distribution

The Hue-Saturation-Value (HSV) colour space separates chromatic content (hue) from intensity (value). The hue wheel shows the distribution of hues across 360 degrees, coloured with the actual hue at each position. Peaks in the distribution reveal the dominant colour families.

For the cats-and-dogs dataset, the strong peak near 0–30 degrees (red-orange) confirms the warm bias. A secondary peak near 200–240 degrees (blue) comes from sky backgrounds and blue collars in dog photos.

HSV Saturation vs Value Scatter

Each image is represented as a point at (mean_saturation, mean_value). Saturation measures colour intensity (0 = grey, 255 = vivid colour); Value measures brightness (0 = black, 255 = white). This scatter reveals four quadrants: bright vivid images (top-right), dark vivid images (bottom-right), bright pastel/grey images (top-left), and dark grey images (bottom-left).

Lab Colour Space (a* vs b*)

The CIE Lab colour space is perceptually uniform — equal numerical distances correspond to equal perceptual differences. The a* axis runs from green (negative) to red (positive); the b* axis runs from blue (negative) to yellow (positive). The plot shows each image as a point in a*/b* space.

For the cats-and-dogs dataset, the cluster in the plot is tightly grouped in the positive-a*, positive-b* quadrant — warm, yellowish-red tones dominating, consistent with fur colours. The tight cluster also indicates low chromatic diversity: most images have similar colour character.

Colour Temperature

Each image is classified as warm (R > 1.1 × B), cool (R < 0.9 × B), or neutral. The pie chart in plot_colour() shows the breakdown. In the cats-and-dogs dataset: 60.4% warm, 35.9% neutral, 3.7% cool — strongly warm overall. This is not a data quality issue but is worth knowing; a model trained on this dataset may struggle with cool-toned test images.

Greyscale-Like Detection

Images with mean HSV saturation below 15 are flagged as greyscale-like. These could be actual greyscale images that were loaded as 3-channel (all channels equal), or naturally desaturated colour images (e.g. winter snow scenes). In the cats-and-dogs dataset, 256 images (1%) were greyscale-like — a small but real population that should be handled by preprocessing.

4.5 Texture Analysis (GLCM)

Texture describes the spatial arrangement of intensities within an image — whether it is smooth (like sky), rough (like fur), regular (like brickwork), or random (like noise). VisEDA uses the Grey-Level Co-occurrence Matrix (GLCM) to compute six standard texture features. The GLCM counts how often pairs of pixel values occur at a given spatial relationship (distance and angle).

Requires scikit-image. Computed at 128×128 resolution with 64 intensity levels for speed. Each feature is averaged across four angles (0°, 45°, 90°, 135°) to achieve rotation invariance.

Contrast

Measures the local variation between adjacent pixels. High contrast = large intensity differences between neighbours. Range: 0 to (number_of_levels − 1)². In the cats-and-dogs dataset, mean GLCM contrast = 31.48. This is relatively high — animal fur and background edges create strong local intensity variation.

Homogeneity (Inverse Difference Moment)

Measures how close the GLCM values are to the diagonal. A homogeneous image has pixels that tend to be similar to their neighbours (smooth regions). Range: 0 to 1 (1 = perfectly homogeneous). The cats-and-dogs dataset shows mean homogeneity = 0.42 — moderate, as expected for images with both textured (fur) and smooth (background) regions.

Energy (Angular Second Moment)

Measures the uniformity of the GLCM. A constant image has energy = 1; a uniformly distributed GLCM has low energy. In practice, natural images have low energy (mean = 0.08 in the cats-and-dogs dataset), while synthetic or highly compressed images may have higher energy.

Correlation

Measures how correlated neighbouring pixels are linearly. Range: −1 to 1 (1 = perfectly correlated). Natural images typically have high correlation (mean = 0.91 in the cats-and-dogs dataset) because nearby pixels tend to belong to the same object and thus have similar values. Very low correlation indicates a noisy or random-textured image.

ASM (Angular Second Moment)

Very similar to energy (some implementations define them identically). Measures the orderliness of the texture. High ASM = the image has a few dominant textures that repeat.

Dissimilarity

Similar to contrast but with a linear rather than quadratic penalty for differences. High dissimilarity means pixels differ substantially from their neighbours; low dissimilarity means the image is mostly smooth.

Reading the Radar Chart

In plot_quality(), the six GLCM features are shown in a radar (spider) chart. Each feature is normalised to its maximum value across the dataset so all axes are on the same scale (0–100%). A large filled area indicates a high average value for that feature. For the cats-and-dogs dataset, the chart shows a spike only on the Contrast axis, with all other features near-zero — meaning the dominant textural property is strong local variation (fur edges), while the images are globally homogeneous and highly correlated.

4.6 Frequency Analysis (FFT)

Every image can be decomposed into sinusoidal waves at different frequencies using the Fast Fourier Transform (FFT). Low-frequency components describe slow-varying regions (gradients, large shapes); high-frequency components describe rapid variation (fine texture, edges, noise). VisEDA divides the FFT spectrum into three concentric rings and measures how much of the total energy each ring contains.

Low Frequency Band (< 10% of max frequency)

Contains the dominant, large-scale structure of the image — sky vs ground, overall lighting gradient, large object shapes. In the cats-and-dogs dataset, mean low-freq energy = 22.9%. This is the smallest band by area but concentrates a large fraction of the image's "story".

Mid Frequency Band (10–40% of max frequency)

Contains medium-scale features — object textures, fur patterns, moderate-sized edges. Mean mid-freq energy = 30.6% for the cats-and-dogs dataset.

High Frequency Band (> 40% of max frequency)

Contains fine detail, noise, and compression artefacts. This is the largest band by area. Mean high-freq energy = 46.5% in the cats-and-dogs dataset. A higher high-freq fraction in a dataset generally means more detailed or noisy images; a lower fraction means smoother, more uniform images.

The bar chart in plot() shows mean ± std for each band. Wide error bars indicate high variability between images — some are very detailed, others very smooth.

4.7 Duplicate Detection

Duplicate images in training data cause the model to effectively see those images more often than others, biasing learning. Duplicates in the overlap between train and validation sets cause data leakage — the model has memorised those images, inflating validation accuracy. VisEDA detects both exact duplicates and near-duplicates.

Exact Duplicates (MD5 Hash)

Two images are exact duplicates if their MD5 checksums match — i.e., they are byte-for-byte identical. This catches images that were literally copied with the same filename or a different filename. For the cats-and-dogs dataset, 0 exact duplicate groups were found — each file is unique at the byte level.

Near-Duplicates (Perceptual Hash)

Two images are near-duplicates if their perceptual hashes (pHash) differ by at most phash_threshold bits. The pHash is computed by resizing the image to 32×32, applying a 2D DCT, taking the 8×8 top-left corner (low-frequency components), and encoding whether each value is above or below the median as a 64-bit integer. Images that are visually very similar — even after resizing, minor colour adjustment, or light JPEG recompression — produce similar pHashes.

For the cats-and-dogs dataset, 194 near-duplicate groups were found. These likely include images that appear in both the Cat/ and Dog/ folders, or images downloaded multiple times at slightly different quality levels.

Action: Review near-duplicate groups with eda.plot_duplicates(mode="near") and decide whether to remove them. If a near-duplicate pair spans train/val splits, remove one image to prevent data leakage.

Chapter 5 — Visualisations

This chapter explains every plot method — what it shows, how to read it, and what to look for in the output.

5.1 plot() — Main EDA Dashboard

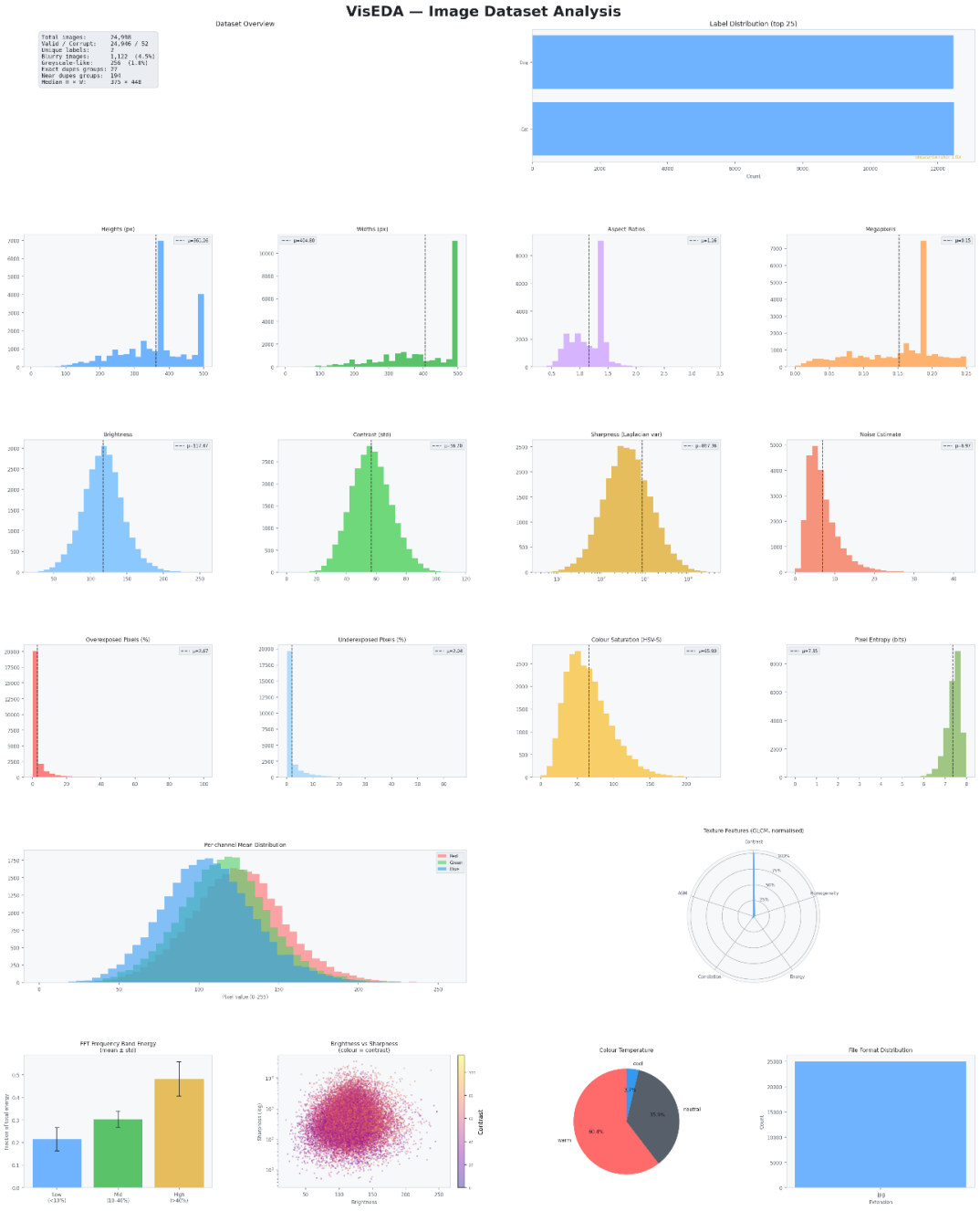

Figure 5.1 — Main EDA dashboard for the cats-and-dogs dataset (24,998 images). Each row covers a different analysis dimension.

The main dashboard, as shown in Figure 5-1, is a six-row, four-column grid of plots that gives a complete overview of the dataset in a single figure. It is the best starting point for any new dataset.

Row 1: Dataset Overview (left) and Label Distribution (right)

The overview card shows the most important dataset facts at a glance: total image count, valid/corrupt breakdown, number of unique labels, blurry and greyscale-like image counts, duplicate group counts, and median image dimensions. This is the first thing to read when opening the dashboard.

The label distribution bar chart shows how many images belong to each class. For the cats-and-dogs dataset, the distribution is nearly perfectly balanced (Cat: 12,479, Dog: 12,467 — imbalance ratio = 1.001), which is ideal. A dataset with an imbalance ratio above 10 would need resampling or class-weighted loss.

Row 2: Spatial Distributions

Four histograms showing heights, widths, aspect ratios, and megapixels. The dashed vertical line marks the mean. In the cats-and-dogs dataset, the large spike at 500 px in the width histogram reveals that many images were capped at 500 pixels — a preprocessing artefact from dataset creation.

Row 3: Quality Metrics

Brightness, contrast, sharpness (log scale), and noise estimate histograms. The sharpness histogram uses a logarithmic x-axis because sharpness values span multiple orders of magnitude — a linear axis would compress most of the distribution into a narrow band near zero, making it unreadable.

Row 4: Exposure and Colour Quality

Overexposed pixel fraction, underexposed pixel fraction, colour saturation, and pixel entropy. The saturation histogram shows a peak around 65–70 (HSV scale 0–255), indicating moderately vivid colours — expected for animal photographs.

Row 5: Channel Histograms (left) and GLCM Radar (right)

The overlapping R/G/B histograms show how pixel value distributions differ between channels. The radar chart shows the six GLCM texture features normalised to their maximum — quickly revealing which textural properties dominate.

Row 6: FFT Energy, Brightness-Sharpness Scatter, Colour Temperature Pie, Format Distribution

The FFT bar chart shows mean ± std for low/mid/high frequency bands. The brightness-sharpness scatter plots each image as a point, coloured by its contrast — revealing correlations between these three quality dimensions. Bright, sharp, high-contrast images cluster in the upper-right; dark, blurry, low-contrast images in the lower-left. The colour temperature pie summarises the warm/neutral/cool split. The format bar shows file format diversity.

5.2 plot_colour() — Colour Analysis Dashboard

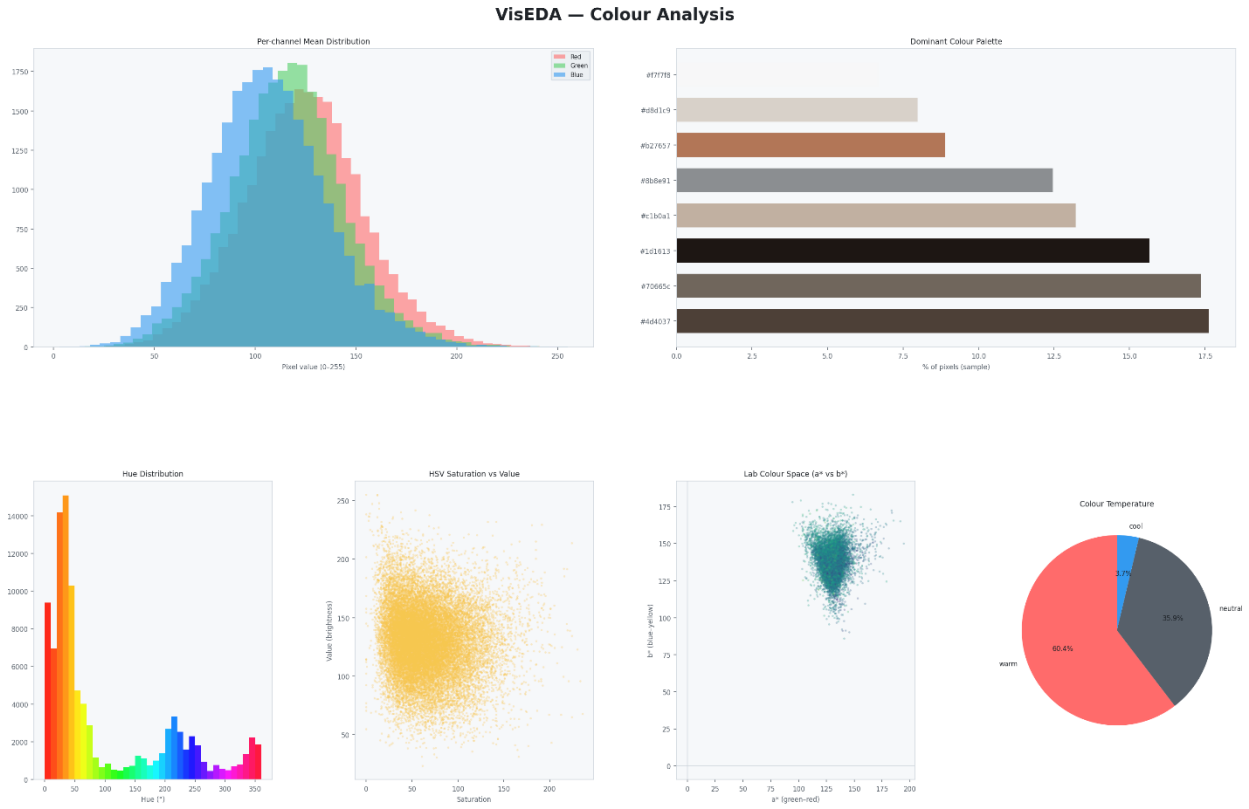

Figure 5.2 — Colour analysis dashboard for the cats-and-dogs dataset.

Per-Channel Mean Distribution (top left)

Overlapping histograms of per-image mean R, G, B values. The rightward shift of the Red histogram relative to Blue confirms the warm colour bias. The tight distribution (low spread) means the colour character is consistent across images — most images have similar average brightness per channel.

Dominant Colour Palette (top right)

The K-Means palette for the cats-and-dogs dataset is dominated by dark browns (#1d1613, #494039, #826454), tans and beiges (#c79e79, #c7bfb7), and near-whites (#f2f1f1) — the natural colours of pet fur and neutral backgrounds. This palette accurately captures the dataset's visual identity.

Hue Distribution (bottom left)

The rainbow-coloured bar chart shows the frequency of each hue angle (0–360°). The dominant peak at 0–30° (red-orange) confirms the warm bias. The secondary peaks at 200–250° (cyan-blue) come from sky backgrounds and water in outdoor photos.

HSV Saturation vs Value (bottom centre)

Each image is a point at (saturation, value). The cats-and-dogs scatter is concentrated in the moderate saturation (50–150), moderate-to-high brightness (100–200) region — expected for well-lit animal photographs. Points near the bottom-left (low sat, low value) would be dark greyscale images.

Lab a* vs b* Scatter (bottom right of centre)

The tight cluster in the positive a* (reddish), positive b* (yellowish) region confirms the warm palette. The cluster is elongated along the b* axis, suggesting more variation in yellow-blue balance than in green-red balance — which makes sense as some images have bluer backgrounds than others.

Colour Temperature Pie

60.4% warm, 35.9% neutral, 3.7% cool. This matches the Lab scatter perfectly — the dataset is predominantly warm-toned.

5.3 plot_quality() — Quality Analysis Dashboard

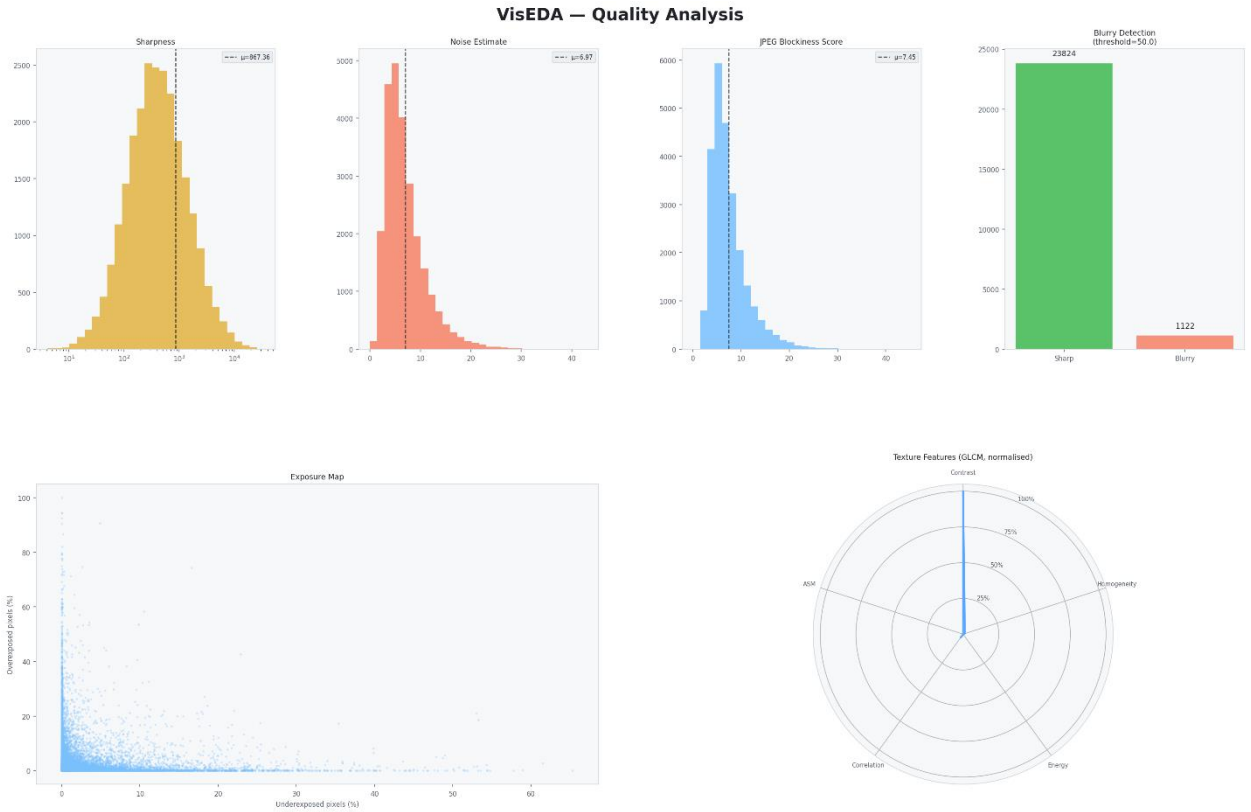

Figure 5.3 — Quality metrics dashboard for the cats-and-dogs dataset.

Sharpness Histogram (top left)

The distribution is right-skewed on a log scale — most images cluster around 100–1000 (reasonably sharp), with a long right tail of very sharp images. Images below 50 (left of the leftmost visible bars) are flagged as blurry. The dashed vertical line at 867 shows the mean — above the blur threshold, indicating the dataset is predominantly sharp.

Noise Estimate (top centre-left)

The narrow, left-skewed distribution peaking around 5–8 indicates consistently low noise — expected for modern camera images. A right tail extending to 40 represents a small number of high-noise images (likely night shots or heavily cropped images).

JPEG Blockiness Score (top centre-right)

Peaks near 5–8, consistent with moderate JPEG compression. The distribution is right-skewed — a small number of heavily-compressed images have high blockiness scores. Very high scores (> 20) would indicate severe quality degradation.

Blurry Detection Bar Chart (top right)

Shows 23,824 sharp images and 1,122 blurry images. The absolute counts make it easy to judge severity: 1,122 blurry images in a 25,000-image dataset is 4.5% — significant enough to warrant either removing the blurry images or testing whether they affect model performance.

Exposure Map (bottom left)

Each image is plotted at (underexposed_fraction, overexposed_fraction). Most images cluster near the origin — well-exposed with minimal clipping. The sparse points with high overexposed fraction (top of the chart) represent images with blown-out backgrounds (e.g., backlit subjects). Points far right (high underexposed fraction) represent very dark images, possibly night shots.

GLCM Radar Chart (bottom right)

The narrow spike on the Contrast axis with near-zero values on all others confirms that local intensity variation (edges and fur texture) is the dominant textural property, while the images are otherwise highly correlated and homogeneous at the GLCM level. This pattern is typical of natural photographic datasets.

5.4 plot_texture() — Texture & Frequency Analysis

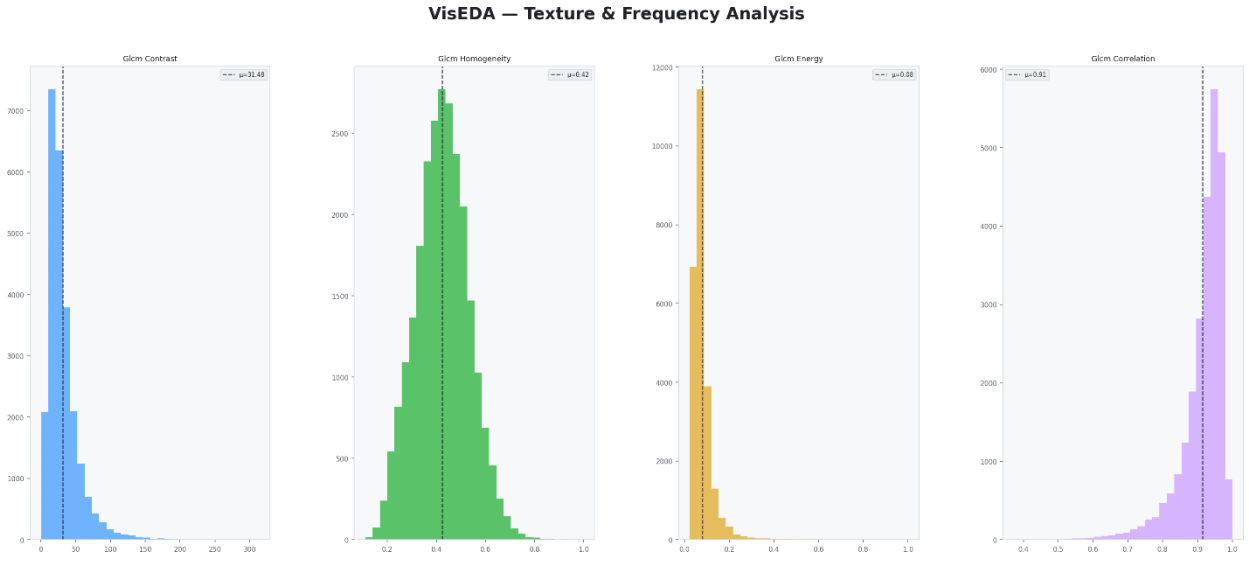

Figure 5.4 — GLCM texture feature distributions for the cats-and-dogs dataset.

Four histograms showing the distribution of GLCM texture features across all images. Each histogram shows a different aspect of texture:

- Contrast (blue, left): Right-skewed distribution peaking near 20–30. Most images have moderate local contrast, with a long tail of very high-contrast images (sharp edges, dramatic lighting).
- Homogeneity (green): Bell-shaped distribution centred around 0.42. Most images have moderate homogeneity — neither perfectly smooth nor perfectly chaotic.
- Energy (gold, third): Strongly left-skewed, peaking near 0.05–0.10. Almost all images have very low energy — confirming that the textures are complex and non-uniform, as expected for natural photographs.
- Correlation (purple, right): Bimodal distribution with a strong peak near 0.90–1.00. High correlation means neighbouring pixels tend to be similar — expected for smooth regions. The secondary peak near 0.80 represents images with more abrupt transitions.

5.5 plot_samples() — Sample Image Grid

```
# 25 random images from the full dataset
eda.plot_samples(n=25, cols=5)

# 10 random images from the Cat class only
eda.plot_samples(n=10, cols=5, label="Cat")

# Reproducible sample (fix random seed)
eda.plot_samples(n=25, random_seed=123)
```

Displays a grid of randomly sampled images. Image titles show the label (or filename stem if no label). This is the most human-interpretable view of the dataset — always look at raw samples before drawing conclusions from statistics.

5.6 plot_class_samples() — Per-Class Sample Grid

```
# 6 samples per class, all classes
eda.plot_class_samples(n_per_class=6)
```

Shows one row per class, making it easy to compare intra-class variety and inter-class similarity visually. For a cats-and-dogs dataset, this confirms that both classes contain diverse breeds, backgrounds, and poses.

5.7 plot_average_image() — Dataset Mean and Variance Image

```
eda.plot_average_image(target_size=(224, 224))

# Per-class average
eda.plot_average_image(target_size=(224, 224), label="Cat")
eda.plot_average_image(target_size=(224, 224), label="Dog")
```

Computes the pixel-wise average of all images (resized to target_size). The left panel shows the mean image; the right shows the pixel-wise standard deviation image.

The mean image reveals systematic dataset biases. If animals are always centred, the mean image will show a blurry animal silhouette in the centre. If backgrounds are consistent (e.g. always green grass), the mean image will show green at the bottom and blue at the top.

The standard deviation image shows where pixel values vary most across the dataset. High-variance regions (bright in the std image) are where the dataset is most diverse. Low-variance regions (dark) are where images are consistently similar — often the background.

5.8 plot_channel_correlation() — R/G/B Scatter Matrix

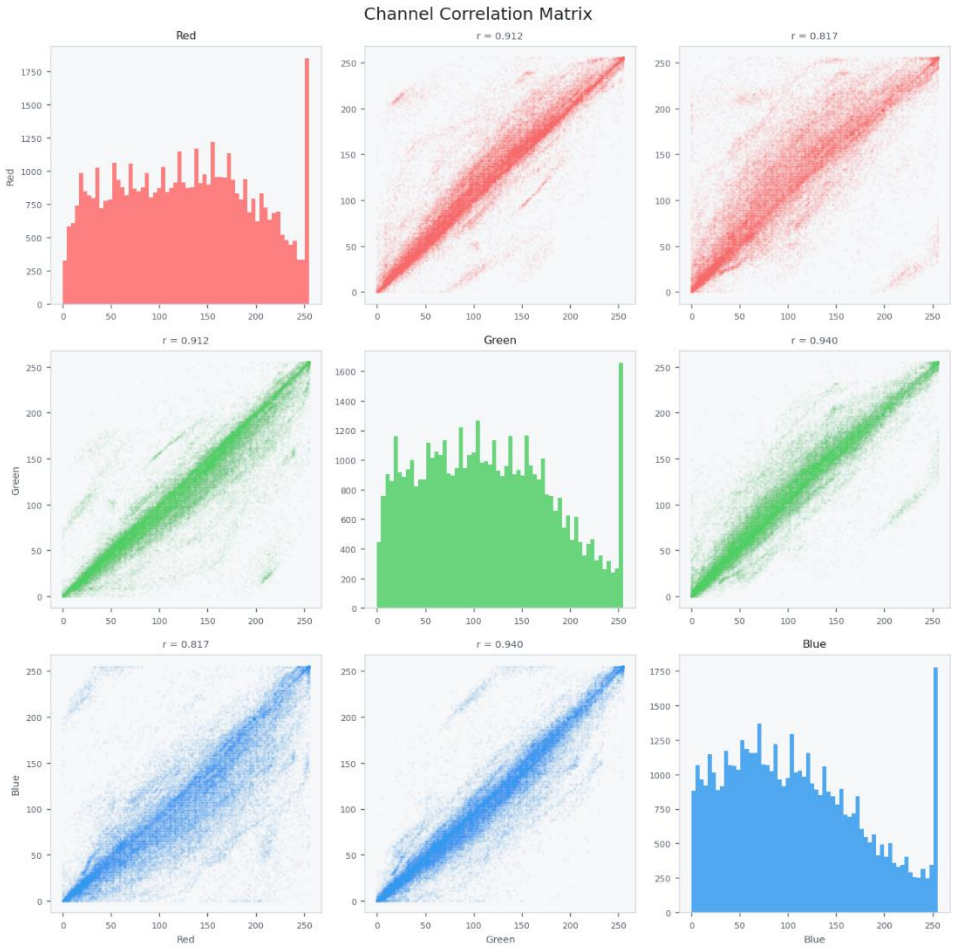

Figure 5.5 — Channel correlation matrix. Diagonal: per-channel pixel histograms. Off-diagonal: pairwise scatter plots with Pearson r values.

A 3×3 grid where the diagonal shows per-channel pixel value histograms and the off-diagonal cells show pairwise scatter plots of pixel values. The Pearson correlation coefficient r is shown above each scatter plot.

For the cats-and-dogs dataset: R-G correlation = 0.912, R-B = 0.817, G-B = 0.940. All three channels are highly correlated — meaning pixels that are bright in one channel tend to be bright in all channels. This is characteristic of natural images, where luminance varies more than chrominance. Near-perfect correlation (r > 0.99) would suggest the images are effectively greyscale.

The elongated diagonal shape of each scatter cloud reflects the overall correlation. A circular cloud would mean no correlation. A narrow line would mean perfect correlation.

5.9 plot_duplicates() — Duplicate Group Viewer

```
# View near-duplicate groups
eda.plot_duplicates(mode="near", max_groups=5)

# View exact duplicates
eda.plot_duplicates(mode="exact", max_groups=5)
```

Displays each duplicate group as a row of images, making it visually obvious why they were flagged. Near-duplicates in the cats-and-dogs dataset include images that appear in both Cat/ and Dog/ folders (mislabelled copies), and images downloaded at different JPEG quality levels from the same source.

Chapter 6 — Advanced Usage

6.1 Working with Large Datasets

For datasets with hundreds of thousands of images, two strategies reduce analysis time:

```
# Strategy 1: Limit the number of images (quick overview)
eda = ImageEDA(max_images=5000)
eda.load("large_dataset/")

# Strategy 2: Skip expensive computations
eda = ImageEDA(compute_glcm=False, compute_freq=False)
eda.load("large_dataset/")

# Strategy 3: Load from a pre-existing list (e.g. from a CSV)
import pandas as pd
df = pd.read_csv("metadata.csv")
paths = df["filepath"].tolist()
labels_dict = dict(zip(df["filepath"], df["label"]))
eda.load(paths, labels=labels_dict)
```

6.2 Accessing Per-Image Records

Every analysed image has an ImageRecord object stored in eda._records. You can filter, sort, and inspect these directly:

```
# Find the 10 blurriest images
blurry = sorted(eda._records, key=lambda r: r.sharpness or 0)[:10]
for r in blurry:
    print(r.path, r.sharpness)

# Find overexposed images
overexp = [r for r in eda._records if r.overexposed_frac > 0.5]
print(f"{len(overexp)} images more than 50% overexposed")

# Find warm images in the Cat class
warm_cats = [r for r in eda._records
             if r.label == "Cat" and r.color_temp == "warm"]

# Export to a DataFrame for custom analysis
import pandas as pd
data = [{
    "path":        r.path,
    "label":       r.label,
    "brightness":  r.brightness,
    "sharpness":   r.sharpness,
    "is_blurry":   r.is_blurry,
    "color_temp":  r.color_temp,
} for r in eda._records if not r.is_corrupt]
df = pd.DataFrame(data)
```

```
df.to_csv("image_stats.csv", index=False)
```

6.3 Using VisEDA in a Training Pipeline

VisEDA integrates naturally into a data preparation workflow:

```
from viseda import ImageEDA
import torchvision.transforms as T
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

# Step 1: Analyse the dataset
eda = ImageEDA(verbose=True)
eda.load("dataset/train/", label_from_parent=True)

# Step 2: Get normalisation statistics
norm = eda.normalization_stats()

# Step 3: Identify problematic images
stats = eda.summary()
print("Blurry images:", stats["quality"]["blurry_count"])
print("Near-dupes:  ", stats["duplicates"]["n_near_duplicate_groups"])

# Step 4: Build transforms using computed stats
transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=norm["mean"], std=norm["std"]),
])

# Step 5: Build the DataLoader as usual
dataset = ImageFolder("dataset/train/", transform=transform)
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

6.4 Comparing Two Datasets

Run VisEDA on both datasets and compare the summary dictionaries:

```
eda_train = ImageEDA(verbose=False)
eda_val   = ImageEDA(verbose=False)

eda_train.load("dataset/train/", label_from_parent=True)
eda_val.load("dataset/val/",     label_from_parent=True)

s_train = eda_train.summary()
s_val   = eda_val.summary()

# Compare key statistics
for key in ["brightness", "contrast", "sharpness"]:
```

```
    m_tr = s_train["quality"][key]["mean"]
    m_va = s_val["quality"][key]["mean"]
    print(f"{key}: train={m_tr:.1f}  val={m_va:.1f}")

# Large differences suggest train/val distribution mismatch
```

Chapter 7 — Interpreting Results and Common Findings

7.1 Red Flags to Watch For

The following findings in VisEDA output should prompt action before training:

- **Blurry fraction > 10%:** Review blurry images manually. Consider removing them or using sharpness as a sampling weight.
- **Class imbalance ratio > 5:** Use class-weighted loss, oversample the minority class, or undersample the majority class.
- **Near-duplicate groups spanning known splits:** Remove duplicates from the smaller split to prevent data leakage.
- **Corrupt files > 1%:** Investigate the source — large numbers of corrupt files suggest a download or storage problem.
- **Greyscale-like fraction > 5% in an RGB dataset:** Convert greyscale images to 3-channel or remove them if the task requires colour.
- **Overexposed fraction > 20% on average:** Check whether exposure correction preprocessing is needed.
- **Very low entropy (< 4 bits) in many images:** May indicate blank, corrupted, or extremely compressed images — review manually.

7.2 Cats-and-Dogs Dataset: Key Findings Summary

Applying VisEDA to the Microsoft Kaggle cats-and-dogs dataset (25,000 images) revealed the following:

- 24,998 total images; 52 corrupt (0.2%). The corrupt images are worth removing.
- Near-perfect class balance (Cat: 12,479, Dog: 12,467 — ratio 1.001). No resampling needed.
- 1,122 images flagged as blurry (4.5%). Worth investigating; removing them may improve model sharpness sensitivity.
- 194 near-duplicate groups found. Recommend reviewing before finalising train/val splits.
- Strongly warm colour distribution (60.4% warm). Models trained on this dataset may underperform on cool-toned test images.
- Dataset normalisation stats: mean=[0.489, 0.452, 0.418], std=[0.230, 0.229, 0.229].
- Mean image size: 361×405 px — safely above the standard 224×224 training resolution with no upscaling needed.

Chapter 8 — API Reference

Complete reference for all public methods of the ImageEDA class.

ImageEDA.__init__()

Parameter	Description
verbose	bool, default True. Print progress messages during loading.
max_images	int | None, default None. Limit the number of images analysed.
n_colors	int, default 8. Number of dominant colour clusters (K-Means).
phash_threshold	int, default 10. Hamming distance threshold for near-duplicate detection.
blur_threshold	float, default 50.0. Laplacian variance below this value = blurry.
compute_glcm	bool, default True. Compute GLCM texture features (needs scikit-image).
compute_freq	bool, default True. Compute FFT frequency band energies.

ImageEDA.load()

Parameter	Description
source	str | Path | List. Directory, single file, or list of paths.
labels	dict | None. Optional {path: label} mapping.
label_from_parent	bool, default False. Infer label from parent folder name.
recursive	bool, default True. Recurse into sub-directories.

ImageEDA.load_arrays()

Parameter	Description
arrays	List[np.ndarray]. Images as HxW or HxWx3, uint8 or float32.
labels	List[str] | None. Optional label for each array.

ImageEDA.summary()

Returns a nested dictionary. Top-level keys: inventory, spatial, pixel_stats, quality, colour, texture, frequency, duplicates, labels. Each numeric statistic is a dict with keys: min, max, mean, median, std, p25, p75.

ImageEDA.normalization_stats()

Returns {"mean": [R, G, B], "std": [R, G, B]} in [0, 1]. Values are averaged per-channel means and stds across all valid images.

ImageEDA.plot()

Parameter	Description
figsize	Tuple, default (24, 28). Figure size in inches.
save_path	str | None. If set, saves to this path instead of showing.
dpi	int, default 150. Resolution for saved figures.

ImageEDA.plot_colour()

Parameter	Description
figsize	Tuple, default (22, 14).
save_path	str | None.
dpi	int, default 150.

ImageEDA.plot_quality()

Parameter	Description
figsize	Tuple, default (22, 14).
save_path	str | None.
dpi	int, default 150.

ImageEDA.plot_texture()

Parameter	Description
figsize	Tuple, default (22, 10).
save_path	str | None.
dpi	int, default 150.

ImageEDA.plot_samples()

Parameter	Description
n	int, default 25. Number of images to display.
cols	int, default 5. Number of columns in the grid.
label	str | None. If set, show only images with this label.
random_seed	int, default 42. Seed for reproducible sampling.
figsize	Tuple | None. Auto-computed if None.
save_path	str | None.

ImageEDA.plot_class_samples()

Parameter	Description
n_per_class	int, default 5. Images per class row.
save_path	str | None.

ImageEDA.plot_average_image()

Parameter	Description
target_size	Tuple, default (224, 224). All images resized to this before averaging.
label	str | None. If set, average only images with this label.
save_path	str | None.

ImageEDA.plot_channel_correlation()

Parameter	Description
max_pixels	int, default 50000. Pixel sample size for scatter plots.
save_path	str | None.

ImageEDA.plot_duplicates()

Parameter	Description
mode	"near" | "exact", default "near".
max_groups	int, default 5. Maximum duplicate groups to display.
save_path	str | None.

ImageEDA.report()

Parameter	Description
output_path	str, default "viseda_report.html". Path for the HTML output file.

Chapter 9 —Roadmap

Version 1.0.0 (May 2026) — Initial Release

- ImageEDA with 20+ per-image metrics.
- HyperspectralEDA and PointCloudEDA (in development).
- HyperspectralDatasetEDA and PointCloudDatasetEDA for collection-level analysis.
- CLI with image, hyper, and cloud sub-commands.
- Self-contained HTML report generation.
- Light theme with configurable dashboards.

Roadmap

- v1.2.0: Video EDA (temporal frame statistics, motion blur detection).
- v1.2.0: Interactive Plotly dashboards as an alternative to Matplotlib.
- v1.3.0: Depth map / RGB-D EDA module.
- v1.3.0: Jupyter widget integration for notebook-first workflows.
- v1.4.0: GPU-accelerated statistics via CuPy.
- v1.4.0: UMAP and t-SNE embedding of image features for dataset exploration.

Appendix A — ImageRecord Fields

Each analysed image is stored as an ImageRecord object. The following fields are available:

Parameter	Description
path	str. Absolute path to the image file, or "<array_N>" for array-loaded images.
label	str | None. Class label if provided.
file_ext	str. File extension (e.g. ".jpg").
file_size_kb	float. File size in kilobytes.
height	int. Image height in pixels.
width	int. Image width in pixels.
channels	int. Number of colour channels (1, 3, or 4).
dtype	str. NumPy dtype string (e.g. "uint8").
aspect_ratio	float. width / height.
megapixels	float. height * width / 1_000_000.
mean_rgb	[R, G, B]. Per-channel mean pixel values (0–255).
std_rgb	[R, G, B]. Per-channel std of pixel values.
brightness	float. Mean luminance (ITU-R BT.601 weighted, 0–255).
contrast	float. Std of luminance across the image.
sharpness	float. Variance of Laplacian filter response.
noise_estimate	float. Std of high-frequency residual after Gaussian blur.
overexposed_frac	float. Fraction of pixels with luminance > 245.
underexposed_frac	float. Fraction of pixels with luminance < 10.
is_blurry	bool. True if sharpness < blur_threshold.
compression_score	float. JPEG blockiness score.
mean_hsv	[H, S, V]. Mean HSV values (H: 0–180, S: 0–255, V: 0–255).
mean_lab	[L, a*, b*]. Mean CIE Lab values.
saturation_mean	float. Mean HSV saturation (0–255).
color_temp	"warm" | "neutral" | "cool". Colour temperature classification.
is_grayscale_like	bool. True if saturation_mean < 15.
entropy_val	float. Shannon entropy of pixel value histogram (bits).

Parameter	Description
glcm_contrast	float | None. GLCM contrast feature (if compute_glcm=True).
glcm_homogeneity	float | None. GLCM homogeneity feature.
glcm_energy	float | None. GLCM energy feature.
glcm_correlation	float | None. GLCM correlation feature.
glcm_asm	float | None. GLCM Angular Second Moment.
glcm_dissimilarity	float | None. GLCM dissimilarity feature.
freq_low	float | None. Low-frequency FFT energy fraction (if compute_freq=True).
freq_mid	float | None. Mid-frequency FFT energy fraction.
freq_high	float | None. High-frequency FFT energy fraction.
phash	int | None. 64-bit DCT perceptual hash.
ahash	int | None. 64-bit average hash.
md5	str | None. MD5 checksum of raw pixel bytes.
is_corrupt	bool. True if the image could not be read.

Appendix B — Glossary

- **GLCM:** Grey-Level Co-occurrence Matrix. A statistical method for examining texture by considering the spatial relationship of pixels.
- **Laplacian:** A second-order derivative image filter that highlights regions of rapid intensity change (edges). Its variance measures overall image sharpness.
- **pHash:** Perceptual hash. A hash computed from the low-frequency DCT components of a small version of the image, robust to minor visual changes.
- **FFT:** Fast Fourier Transform. Decomposes an image into sinusoidal components at different spatial frequencies.
- **HSV:** Hue-Saturation-Value. A colour space that separates chromatic content (hue), colour purity (saturation), and brightness (value).
- **Lab:** CIE L*a*b*. A perceptually uniform colour space. L=lightness, a*=green-red axis, b*=blue-yellow axis.
- **ITU-R BT.601:** The standard formula for converting RGB to luminance: L = 0.299R + 0.587G + 0.114B, reflecting human visual sensitivity to each channel.
- **Normalisation stats:** Per-channel mean and standard deviation used to standardise pixel values for neural network training.
- **Data leakage:** When information from the validation or test set influences training, usually by having identical or near-identical images in both splits.

Appendix C — Acknowledgements

VisEDA builds on the excellent work of the following open-source projects: OpenCV (image reading and colour transformations), NumPy (array operations), Matplotlib (visualisations), scikit-image (GLCM texture analysis), scikit-learn (K-Means dominant colour extraction), and Pillow (image format support).

The cats-and-dogs dataset used throughout this book is the Microsoft Kaggle Cats and Dogs Dataset, available at https://www.microsoft.com/en-us/download/details.aspx?id=54765.